

# مستند پروژه مدیریت پارکینگ با Queue و Stack

## 1. مقدمه

پروژه مدیریت پارکینگ با Queue و Stack یک شبیه‌سازی ساختارمند از نحوه‌ی پارک و مدیریت خودروها در یک پارکینگ چندرطبقه است. در این پروژه سعی شده رفتار واقعی پارکینگ‌ها با استفاده از دو ساختار داده مهم شبیه‌سازی شود:

- Stack برای هر ردیف پارکینگ (ورود و خروج از یک سمت)
- Queue برای صف ورودی خودروها

## 2. معماری و ساختار کلی سیستم

### 2.1 کلاس‌ها و نقش آن‌ها

#### 1. کلاس Car

نماینده یک خودرو در سیستم.

- ویژگی: carId — شناسه خودرو
- متد اصلی: getCarId()

#### 2. کلاس Node

گروهی برای ساخت LinkedList مورد استفاده در Queue و Stack.

- ویژگی‌ها:
- data — شی Car
- next — اشاره‌گر به گره بعدی
- متدها: getNext(), getData(), setNext()

### 3. کلاس LinkedListStack (نماینده یک ردیف پارکینگ)

Stack پیاده‌سازی شده با LinkedList.

- ویژگی‌ها:
- top
- capacity
- size
- متدها:

- `push()` — پارک خودرو
- `pop()` — خروج خودرو از بالاترین موقعیت
- `isEmpty()` , `isFull()`

#### 4. کلاس `SimpleLinkedListQueue` (صف ورودی)

یک `Queue` ساده بر اساس `LinkedList`.

- ویژگی‌ها: `size` , `rear` , `front`
- متدها: `isEmpty()` , `peek()` , `dequeue()` , `enqueue()`

#### 5. کلاس `ParkingLot` (مدیریت اصلی پارکینگ)

این کلاس تمام عملیات اصلی سیستم را مدیریت می‌کند.

- ویژگی‌ها:
- آرایه‌ای از `Stack`ها
- صف ورودی
- متدهای مهم:
- `addToQueue(car)`
- `parkFirstAvailable()`
- `parkSpecificStack(i)`
- `findCar(id)`
- `exitCar(id)`
- `transferStacks(i, j)`
- `sortStack(index)`

### 3. توضیح الگوریتم‌ها

#### 3.1 ورود خودرو به پارکینگ

1. خودرو ابتدا وارد صف ورودی می‌شود.
2. با توجه به دستور، یا به اولین `Stack` دارای ظرفیت خالی منتقل می‌شود، یا به یک `Stack` مشخص.

#### 3.2 جستجوی خودرو

- ابتدا کل `Stack`ها پیمایش می‌شوند.
- در هر `Stack`، خودرو از بالا به پایین جستجو می‌شود.
- محاسبه موقعیت از پایین:

```
positionFromBottom = capacity - positionFromTop + 1
```

### 3.3 خروج خودرو

- اگر خودرو در بالای Stack باشد: به سادگی خارج می‌شود.
- اگر در میانه باشد: خودروهای بالایی به صورت موقت جابجا می‌شوند تا خودرو آزاد شود.

### 3.4 انتقال خودرو بین Stackها

- خودروها از Stack مبدأ به مقصد منتقل می‌شوند.
- اگر Stack مقصد پر باشد، عملیات به Stackهای بعدی منتقل می‌شود.
- در صورت پر بودن تمام Stackها، پیام مناسب چاپ می‌شود.

### 3.5 مرتب‌سازی Stack

- با Merge Sort بازگشتی و بدون ایجاد آرایه اضافی مرتب‌سازی انجام می‌شود.
- این الگوریتم برای LinkedList بهینه است.

## 4. نمونه خروجی برنامه

```
--- Initializing Parking (3 stacks, capacity = 3) ---
Car 101 added to queue.
Car 102 added to queue.

The Car: 101 in stack number 1 was parked.
The Car: 102 in stack number 1 was parked.

Car 102 found in Stack 1, position from top: 2
Car 999 not found.

Attempting to exit Car 101...
Car 101 exited successfully.

Transferring from Stack 2 to Stack 3...
Car 202 is now in Stack 3.

Sorting Stack 3...
Sort operation complete.
```

## 5. محدودیت‌ها و قوانین

- ظرفیت هر Stack محدود است.
- خودروها حتماً باید ابتدا وارد **صف ورودی** شوند.
- خروج خودرو باید مطابق قوانین LIFO باشد.
- همه عملیات بدون استفاده از کتابخانه‌های آماده Stack و Queue انجام شده‌اند.

## 6. نکات تکمیلی

- سیستم کاملاً قابل توسعه است.
- می‌توان ویژگی‌هایی مانند:
- محاسبه هزینه پارکینگ
- ثبت زمان ورود
- گزارش‌گیری
- ذخیره اطلاعات در فایل را به راحتی اضافه کرد.

## پایان مستند

کاری از ارمیا رستم زاده و طاهرا راستین